

Software Requirements Specification (SRS)

Project Name: ATM Simulation System

Phase: Week 2 Practice Project

Week 2 Learning Goals

This project directly supports the Week 2 roadmap checklist. It is designed to practice:

- ExceptionHandling (try-catch-finally, throw, throws)
- Custom Exceptions
- Exception Propagation
- Interfaces and Abstract Classes
- Multiple Inheritance through Interfaces
- Collections

1. Objective

Create a Java console-based ATM Simulation System that manages bank accounts in memory. The application must demonstrate proper exception handling, interface/abstract class design, and collection usage while simulating real-world ATM operations.

2. Actors

- **User:** Person operating the ATM console application.
- **System:** Validates operations, maintains account records, and handles exceptions.

3. Functional Requirements (FR)

FR-1: Create Account

The user can create a new bank account. **Required Data:**

- Account Number
- Account Holder Name
- Initial Balance

FR-2: View Account

Display complete account details using the account number.

FR-3: Deposit Money

Deposit a valid amount into an account.

FR-4: Withdraw Money

Withdraw money from an account if sufficient balance exists.

FR-5: Transfer Funds

Transfer money between two existing accounts.

FR-6: Check Balance

Display current account balance.

FR-7: Display Total Accounts

Show the number of accounts currently stored in the system.

4. Non-Functional Requirements (NFR)

- **NFR-1:** Console-based interface only.
- **NFR-2:** No database integration.
- **NFR-3:** No file handling.
- **NFR-4:** All data must remain in memory during execution.
- **NFR-5:** Invalid operations must be handled using exceptions.
- **NFR-6:** Application should not terminate unexpectedly due to user errors.

5. Required Class Design

Interface: ATMOperations

Methods:

- deposit(double amount)
- withdraw(double amount)
- checkBalance()

Abstract Class: BankAccount

Attributes:

- private int accountNumber
- private String accountHolderName
- protected double balance

Constructors:

- BankAccount()
- BankAccount(int accountNumber, String accountHolderName, double balance)

Concrete Class: SavingsAccount

Extends BankAccount and implements ATMOperations.

ATMService : Stores accounts using: HashMap<Integer, BankAccount>

6. Implementation Requirements

The project must include the following custom exceptions:

- InvalidAmountException
- InsufficientBalanceException
- AccountNotFoundException

Use HashMap as the in-memory database.

Example: HashMap accounts; Key = Account Number ,Value = Account Object

7. Validation Rules

- The account number must be unique.
- Initial balance cannot be negative.
- Deposit amount must be greater than zero.
- Withdrawal amount must be greater than zero.
- Withdrawal amount cannot exceed balance.
- Transfer amount must be greater than zero.
- Source and destination accounts must exist.

8. User Interface (Console Menu)

===== ATM SIMULATION SYSTEM =====

1. Create Account
2. View Account
3. Deposit Money
4. Withdraw Money
5. Transfer Funds
6. Check Balance
7. Show Total Accounts
8. Exit

9. Success Criteria Checklist

You have successfully completed this phase if your project contains:

- ☐ Uses at least 3 Custom Exceptions
- ☐ Uses HashMap Collection
- ☐ Uses Interface Design
- ☐ Uses Abstract Class Design
- ☐ Uses Encapsulation
- ☐ Uses Inheritance
- ☐ Uses Polymorphism
- ☐ Demonstrates throw
- ☐ Demonstrates throws
- ☐ Demonstrates Exception Propagation
- ☐ Demonstrates try-catch-finally
- ☐ Interactive Console Menu ☐ Clean OOP Design